CEE 292: November 6th Project

Jacky Hua

The paper and the code are linked respectively:

Paper: `https://arxiv.org/pdf/2406.09071`
GitHub: `https://github.com/jackyh3447-coder/Numerical-Solvers-CPP/tree/main/Weak-Form-Flame-PINN`

Forward Solve

We introduce the PDE:

$$0 = -\lambda \frac{\mathrm{d}^2 T}{\mathrm{d}x^2} + \rho_{in} s_L c_P \frac{\mathrm{d}T}{\mathrm{d}x} - \omega q_F \tag{1}$$

Where $\rho_\in$ is the inlet density of a Hydrogen-Oxygen flame mixture, s_L is the laminar flame speed, c_P is the specific heat at constant pressure, ω is the reaction rate, and q_F is the fuel calorific value. Many of these values are given as constants for this particular mixture, and most importantly, ρ, u, p, Y_F, ω are all functions of T. In this section, I aim to do a forward solve of this ODE written in the weak-form. There are two boundary conditions that I will use:

$$T(0) = 298\text{K}, \ \left.\frac{\mathrm{d}T}{\mathrm{d}x}\right|_{x=0} = 10^5 \ \frac{\text{K}}{\text{m}} \tag{2}$$

There is no boundary condition at the end of the flame tail. We have to impose a Neumann Boundary condition because in this problem, one of the parameters, s_L is unknown, and we will be solving it along with the ODE simultaneously in our PINN. The first step is find the weak-form of the problem. We call ϕ the test function here.

$$0 = -\lambda \int_0^L T''\phi \ \mathrm{d}x + \int_0^L \rho_m s_L c_p T' \phi \ \mathrm{d}x - \int_0^L \omega q_F \phi \ \mathrm{d}x \tag{3}$$

$$0 = -\lambda [T'(x)\phi]_0^L + \lambda \int_0^L T'(x)\phi'(x) \ \mathrm{d}x + \lambda \int_0^L T''(x)\phi \ \mathrm{d}x \tag{4}$$

Because the final temperature is allowed to be anything, I can set $\phi(L) = 0$, however, because we have to satisfy a gradient at $x = 0$, we can't make any informed decisions about $\phi(0)$, so just leave it as is. The weak form is therefore:

$$R(s_L) = \lambda \cdot 10^5 \phi(0) + \lambda \int_0^L T'(x)\phi'(x) \ \mathrm{d}x + \int_0^L \rho_m s_L c_P T'(x)\phi(x) \ \mathrm{d}x - \int_0^L \omega q_F \phi \mathrm{d}x \tag{5}$$

Where $R(s_L)$ represents our residual for our loss in our PINN. Assume we use j test functions, our loss for our residual is given by:

$$\mathcal{L}_{weak} = \frac{1}{N} \sum_{j=1}^{N} R_j^2 \tag{6}$$

We can enforce the inlet temperature boundary error as:

$$\mathcal{L}_{in} = (\hat{T}(0) - 298)^2 \tag{7}$$

And as mentioned previously, there's an inlet Neumann condition we need to satisfy

$$\mathcal{L}_{Neu} = \left(\left.\frac{\mathrm{d}T}{\mathrm{d}x}\right|_{x=0} - 10^5 \right)^2 \tag{8}$$

$$\mathcal{L}_{tot} = \mathcal{L}_{weak} + \mathcal{L}_{in} + \mathcal{L}_{Neu} \tag{9}$$

Inverse Flame Speed

Finally, remember that s_L is also a trainable parameter, so we make a guess for s_L initial, and we can update with the optimizer:

$$\theta_{new} = \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}, \; s_{L,new} = s_L - \eta \frac{\partial \mathcal{L}}{\partial s_L} \tag{10}$$

This completes my implementation of the hybrid forward-inverse problem. In my original proposal, I stated I was also going to learn λ and E_a, the thermal conductivity and the activation energy as well, but I understand now those are given parameters. Using the BVP reference solution to manufacture data and find those values using the inverse problem is possible, however given the complexity of even implementing s_L and the fact that in real life, we wouldn't know what the BVP solution is, I elected not to do that.

Results:

Two models were trained - a uniform and a clustered model. The uniform model uniformly distributes collocation points, and the clustered model adds more collocation points toward $x = 0$ to attempt to capture the flame front. The initial guess for s_L was the same for both models.

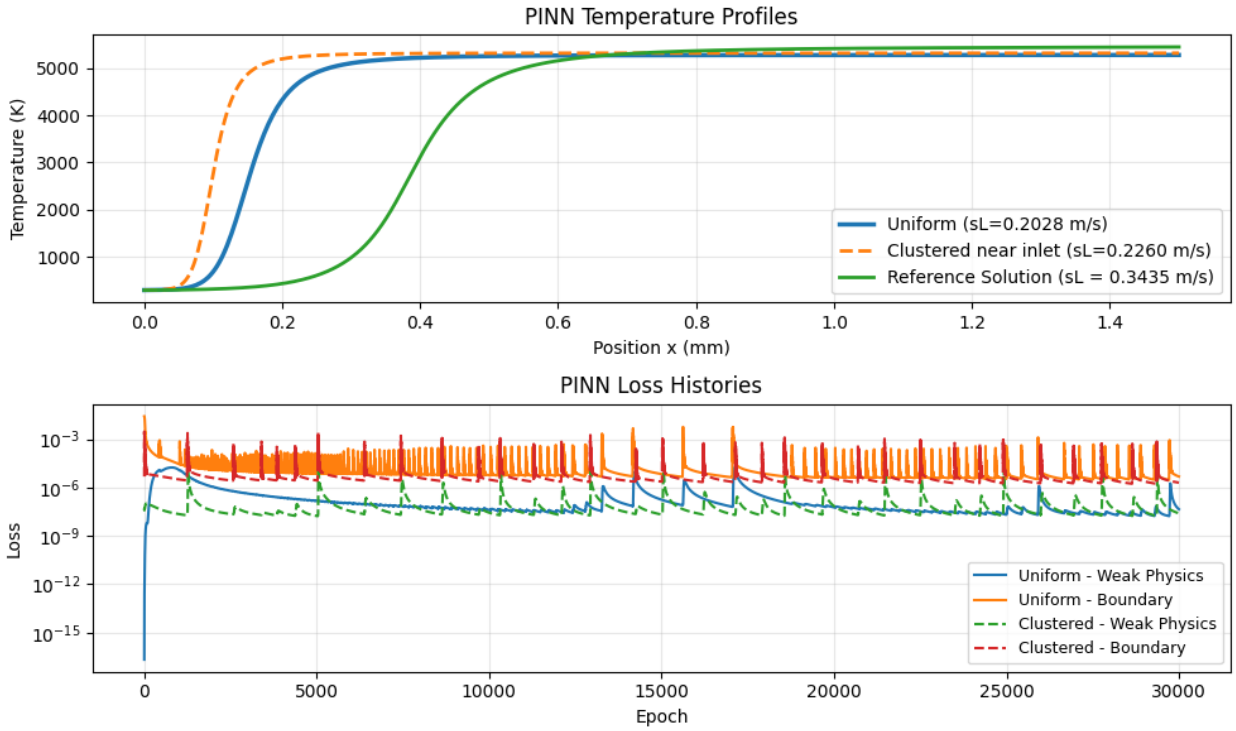


Figure 1: Like the paper, these tests were run at 30000 epochs, 1000 collocation points, 64 neurons, and 3 hidden layers. I decided to set the learning rate to 10^{-3} using an Adam Optimizer, and had 10 test functions for my weak formulation. Each implementation took roughly 5 minutes to run.

Discussion:

There were many challenges running a PINN on a PDE where the learned values are constrained. First, unlike a normal PINN where learned parameters can take on any value, temperature and flame speed have to remain bounded by certain quantities, specifically:

$$s_L > 0, \; T(0) \leq \hat{T} \leq T_{ad} \tag{11}$$

For $Y_{F,in}$, the initial mass fraction, q_F, the calorific fuel value, and c_p, the heat capacity as constants:

$$T_{ad} = \frac{Y_{F,in} q_F}{c_p} + T(0) \tag{12}$$

When training, the flame speed can become negative and the temperature can fall outside of the bound, so techniques like normalizing the temperature and grid so temperature always falls within the bound, and soft constraining s_L using the softplus Python package was required for the training to even occur. The sigmoid temperature distribution specifically causes problems with the boundary condition: sigmoid can never hit $T(0) = T_0$ exactly to satisfy the left boundary condition, so the boundary error seems to plateau at 5×10^{-6}. The physics loss is actually very well-formulated - it naturally decays to roughly the order of 10^{-8} quickly, so most of the correction that happens to s_L is actually just in the boundary error. Because the boundary error seems to stay fixed across all 30000 epochs, I infer that as s_L is learning incorrect values as it attempts to try to change to lower the total loss. In an attempt to make the physics loss and the boundary loss the same magnitude, I multiplied the physics loss by a flat $\alpha = 1000$, and in training, the physics loss truly gets orders of magnitude smaller as epochs progress, however it forces s_L to be incredibly small. Finally, because it's not known where the flame front is ahead of time, the clustering, which has the formulation:

$$\mathrm{x}_{\mathrm{colloc}} = \mathrm{x}^2_{\mathrm{uniform}} \tag{13}$$

Pushed all the collocation points closer to $x = 0$ instead of $x = 0.5$ where the flame front is actually predicted. It noticeably pushed the flame front closer to 0. The formulation above works because x was required to be normalized between 0 and 1 to match the normalized temperature.

What is successful about my implementation, however, is the fact that we are able to even come up with a solution that contains a flame front in the first place! The authors in the paper used a strong-formulation PINN to learn T and s_L, and because those parameters are so particular, the only solution that the PINN could learn was the trivial flat solution when $T(x) = 298$, representing a hot stream of gas that never combusts (Fig 6 in their paper). The authors had to implement a "warm-up" sequence where the PINN starts with a temperature distribution that already looks close to the solution. In my formulation, because more solutions are allowed to satisfy the weak form, we can find alternate solutions that contain a flame front - that was actually the motivation of picking this paper. Not only that, there are actually no loss terms in my formulation that force the tail of the flame to converge a particular value or slope, so it's remarkable to see just how close the final conditions of the flame resemble that of the solution.

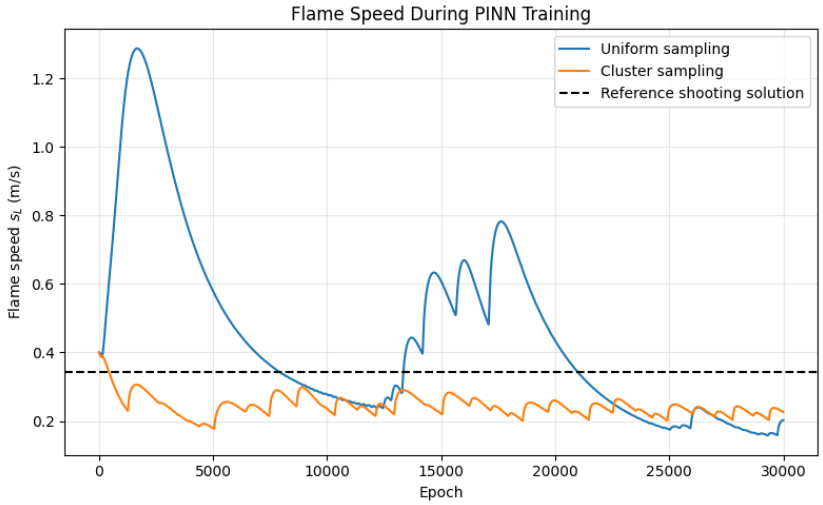


Figure 2: A final graph of how s_L converged across epochs is shown. The clustered collocation points actually did a much better job at approximating the reference shooting solution, however because of the constant BC error, not much more can be said.